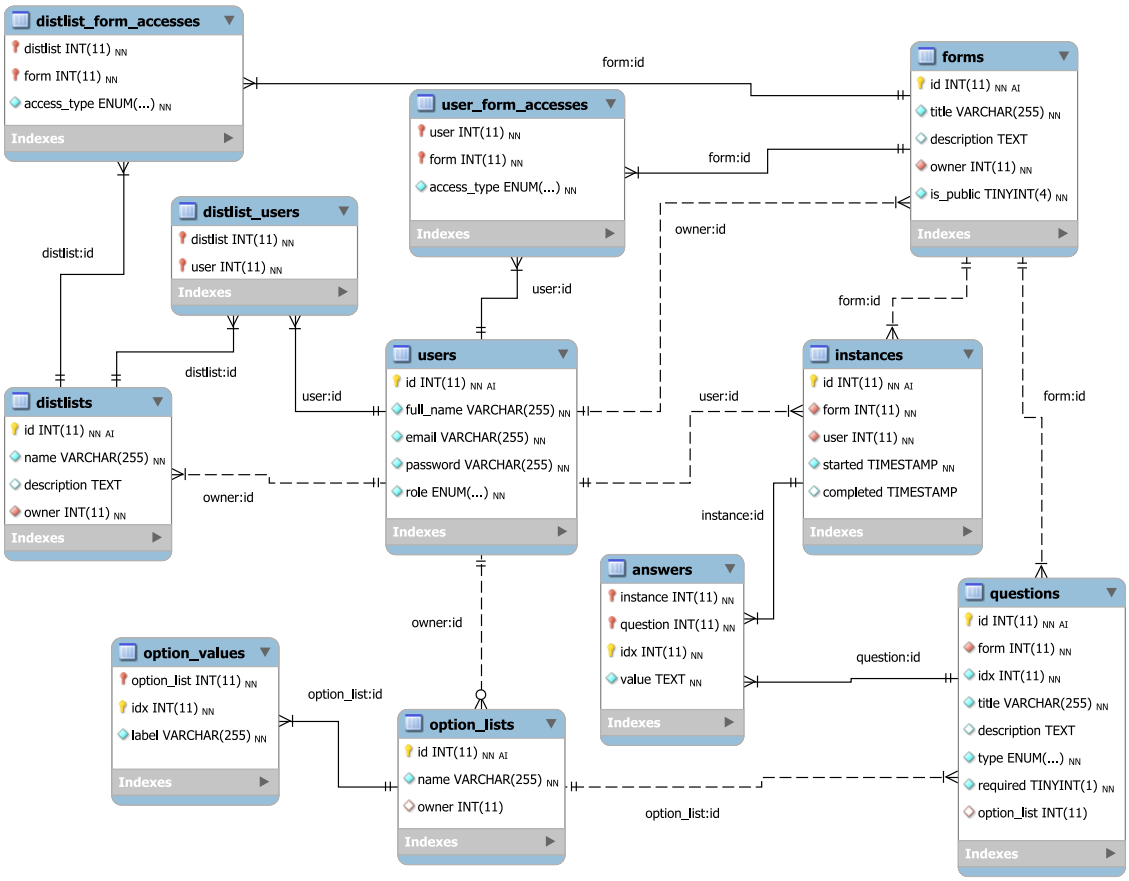
Données et validations métier

Schéma de la base de données

Le schéma de la base de données vous est imposé. Le script SQL de création se trouvera dans votre dépôt BitBucket. Aucune modification de ce schéma n'est autorisée (pas d'ajout, modification ou suppression des tables, champs, index, vues, procédures stockées, triggers, etc).



Remarques générales :

- les colonnes marquées par `NN` sont des colonnes `NOT NULL` ;
- les colonnes marquées par `AI` sont des clés primaires autoincrémentées ;
- la taille maximale des champs de type `VARCHAR` est régie par la base de données sur base de la définition des colonnes correspondantes dans le schéma ; elle doit faire l'objet d'une validation explicite dans votre application.

Le script de création de la base de données insère plusieurs utilisateurs dans la table `users`, dont voici les *credentials* :

E-mail	Mot de passe	Rôle
`guest@epfc.eu`	N/A[1]	`guest`
`admin@epfc.eu`	`Password1,`	`admin`
`bepenelle@epfc.eu`	`Password1,`	`user`
`boverhaegen@epfc.eu`	`Password1,`	`user`
`mamichel@epfc.eu`	`Password1,`	`user`
`xapigeolet@epfc.eu`	`Password1,`	`user`
`student1@epfc.eu`	`Password1,`	`user`
`student2@epfc.eu`	`Password1,`	`user`
`student3@epfc.eu`	`Password1,`	`user`

Description des tables

- La table **`users`** contient les **utilisateurs** du système, caractérisés par un identifiant unique (`id`), un e-mail (`email`), un mot de passe (`password`), un nom complet (`full_name`) et un rôle (`role`).

  Il y a trois types d'utilisateurs : l'utilisateur anonyme (colonne `email` à `'guest@epfc.eu'` et colonne `role` à `'guest'`), les utilisateurs simples (colonne `role` à `'user'`) et les administrateurs (colonne `role` à `'admin'`).

- La table **`forms`** contient les **formulaires**, caractérisés par un identifiant unique (`id`), un titre (`title`), une description (`description`) et un flag indiquant si le formulaire est public (`is_public`).

  Un formulaire est lié à un utilisateur qui est son créateur / propriétaire (colonne `owner`).

- La table **`questions`** contient les **questions** des formulaires. Une question est caractérisée par un identifiant unique (`id`), un ordre d'apparition dans le formulaire (`idx`), un titre

(`title`), une description (`description`), un type de question (`type`) et un flag indiquant si une réponse est requise (`required`).

Les différents types de question sont les suivants :

- `short` : un simple champ d'entrée de texte court sur une seule ligne.
- `long` : un champ d'entrée de texte long sur plusieurs lignes.
- `date` : un champ d'entrée pour une date.
- `email` : un champ d'entrée pour une adresse mail.
- `check` : un ensemble de cases à cocher (*checkboxes*) pour choisir une ou plusieurs valeurs dans une liste d'options.
- `combo` : une liste déroulante (*combobox*) pour choisir une seule valeur dans une liste d'options.
- `radio` : un ensemble de boutons radio (*radio buttons*) pour choisir une seule valeur dans une liste d'options.

On notera que le type de question `check` est le seul qui permette à l'utilisateur de choisir plusieurs valeurs.

La colonne `form` contient l'identifiant du formulaire auquel appartient la question.

La colonne `option_list` contient l'identifiant d'une liste d'options associée à la question ; cette colonne sera différente de `null` dans les cas où le type de question requiert l'utilisation d'une liste d'options, c'est-à-dire pour les types `radio` , `check` et `combo` .

- La table `instances` contient les **instances** des formulaires. Une instance d'un formulaire est créée lorsqu'un utilisateur ouvre ce formulaire pour répondre aux questions. Une instance est caractérisée par un identifiant unique (`id`), une date/heure de début (`started`) et une date/heure de fin (`completed`).

  La colonne `user` contient l'identifiant de l'utilisateur qui a ouvert le formulaire. La colonne `form` contient l'identifiant du formulaire concerné par l'instance.

  La date/heure de début est initialisée avec le moment de la création de l'instance.

  La date/heure de fin, si elle n'est pas nulle, correspond au moment où l'instance du formulaire a été soumise par l'utilisateur. Si elle est nulle, cela signifie que l'instance du formulaire est toujours en cours (n'a pas été soumise). Une fois qu'une instance a été soumise, elle ne peut plus être modifiée (ni son contenu, c'est-à-dire les réponses qu'elle contient).

- La table `answers` contient les **réponses** fournies par les utilisateurs aux questions des formulaires. Une réponse est caractérisée par l'identifiant de l'instance à laquelle elle appartient (`instance`) et l'identifiant de la question à laquelle elle se rapporte (`question`). La colonne `value` contient la valeur de la réponse fournie par l'utilisateur.

  La colonne `idx` contient `0` dans le cas d'une question qui n'est pas liée à une liste d'options. Dans le cas contraire, elle contient l'index de l'option choisie par l'utilisateur (voir

table `option_values`). Dans le cas d'une question de type `check`, où l'utilisateur peut cocher plusieurs valeurs, on créera dans cette table un enregistrement pour chaque option cochée, en mettant dans `idx` à chaque fois l'index de l'option cochée.

La clé primaire de cette table est composée des colonnes `instance`, `question` et `idx`.

- La table **`option_lists`** contient les **listes d'options** définies dans le système. Une liste d'option est caractérisée par un identifiant unique (`id`) et un nom (`name`).

  La colonne `owner` contient l'identifiant de l'utilisateur qui a créé la liste d'options ; seul cet utilisateur (ou un admin) peut utiliser cette liste d'options lorsqu'il crée ou édite des questions. Si cette colonne est nulle, cela indique que la liste d'options est de type "Système" et peut dès lors être utilisée par n'importe quel utilisateur lorsqu'il crée ou édite des questions.

- La table **`option_values`** contient les **options** qui constituent les listes d'options définies dans le système. Une option est caractérisée par l'identifiant de la liste d'option à laquelle elle appartient (`option_list`), la position de l'option au sein de la liste (`idx`) et le libellé associé à l'option (`label`).

  La colonne `idx` permet de numéroter les options au sein d'une liste d'options. Cette numérotation commence à `1` pour la première option de la liste et s'incrémente de `1` pour chaque option supplémentaire. Lors de l'affichage, les options sont présentées dans l'ordre de numérotation. En outre, c'est cette valeur de `idx` qui est stockée dans la colonne `value` de la table `answer` quand le type de question implique une liste d'options.

  La clé primaire de cette table est composée des colonnes `option_list` et `idx`.

- La table **`user_form_accesses`** contient les informations permettant de **partager des formulaires avec d'autres utilisateurs**. Grâce à cette table, le propriétaire d'un formulaire (ou un admin) peut donner accès à son formulaire à d'autres utilisateurs. Un partage (ou "accès") est caractérisé par l'identifiant du formulaire (`form`), l'identifiant de l'utilisateur avec lequel on veut partager le formulaire (`user`) et le type d'accès qu'on veut donner (`access_type`).

  Le type d'accès peut prendre une des valeurs suivantes :

  - `user` : indique que l'utilisateur ciblé ne pourra accéder au formulaire que pour y répondre.
  - `editor` : indique que l'utilisateur ciblé pourra accéder au formulaire avec les mêmes droits que le propriétaire, à l'exception des droits de gestion des partages (voir section "Droits d'accès").

  La clé primaire de cette table est composée des colonnes `user` et `form`.

- La table **`distlists`** contient les **listes de distribution**. L'idée d'une liste de distribution est de pouvoir créer une liste d'utilisateurs sur mesure et de lui donner un nom, puis de pouvoir partager un formulaire avec cette liste, provoquant de ce fait le partage du formulaire avec tous les utilisateurs de la liste en question. Grâce à ce type de liste, le propriétaire d'un formulaire peut le partager en un coup avec un ensemble d'utilisateur préalablement défini.

Une liste de distribution est caractérisée par un identifiant unique (`id`), un nom (`name`), une description (`description`) et l'identifiant d'un utilisateur qui en est le propriétaire.

- La table **`distlist_users`** permet d'indiquer les **utilisateurs qui sont membres des listes de distribution**. Il s'agit donc d'une simple table d'association entre les tables `distlists` (via colonne `distlist`) et `users` (via la colonne `user`).
- La table **`distlist_form_accesses`** contient les informations permettant de **partager des formulaires avec des listes de distribution**. Grâce à cette table, le propriétaire d'un formulaire (ou un admin) peut donner accès à son formulaire à une liste de distribution (et, par là, à ses membres). Un partage (ou "accès") est caractérisé par l'identifiant du formulaire (`form`), l'identifiant de la liste de distribution avec laquelle on veut partager le formulaire (`distlist`) et le type d'accès qu'on veut donner (`access_type`). Les types d'accès sont les mêmes que ceux définis plus haut pour la table `user_form_accesses` .

Validations métier

Voici quelques contraintes métier complémentaires à celles imposées par le schéma de la base de données ci-dessus :

Table `users` :

- `id` est auto-incrémenté.
- `full_name` doit avoir au minimum une longueur de 3 caractères.
- `email` doit avoir le format d'une adresse mail valide ; il doit être unique.
- `password` doit contenir la version *hachée* du mot de passe de l'utilisateur. Ce dernier doit avoir au minimum une longueur de 8 caractères, doit contenir au moins un chiffre, une lettre majuscule et un caractère non alphanumérique.
- `role` contient le rôle de l'utilisateur (`admin` , `user` ou `guest`). Il doit y avoir un et un seul record ayant le rôle `guest` (voir données associées ci-dessus). Il s'agit d'un utilisateur "virtuel" avec lequel on ne doit pas pouvoir se connecter dans l'application. Il sera utilisé en base de données pour représenter n'importe quel utilisateur de l'application qui l'utilise de manière anonyme (sans être connecté à un compte utilisateur) pour accéder à des formulaires publics et y répondre.

Table `forms` :

- `id` est auto-incrémenté.
- `title` doit avoir au minimum une longueur de 3 caractères. Le titre doit être unique pour une même `owner` .
- `description` , si elle est remplie, doit avoir au minimum une longueur de 3 caractères.

Table `instances` :

- `id` est auto-incrémenté.
- `completed`, si elle est remplie, doit être postérieure à `started` et inférieure ou égale à la date/heure courante.

Table `questions` :

- `id` est auto-incrémenté.
- `idx` doit être supérieur à `0`.
- Le couple `form`, `idx` doit être unique, autrement dit deux questions d'un même formulaire ne peuvent pas avoir le même indice.
- `title` doit avoir au minimum une longueur de 3 caractères.
- Le couple `form`, `title` doit être unique, autrement dit deux questions d'un même formulaire ne peuvent pas avoir le même titre.
- `description`, si elle est remplie, doit avoir au minimum une longueur de 3 caractères.
- `required` doit prendre la valeur `0` (faux) ou `1` (vrai).
- `option_list` doit référencer une liste d'options si `type` vaut `'check'`, `'combo'` ou `'radio'`, sinon doit être `null`.

Table `answers` :

- `idx` doit valoir `0` si le type de question est différent de `'check'`, sinon il doit être supérieur à zéro et les différents records qui ont les mêmes valeurs de `instance` et `question` doivent avoir des valeurs consécutives commençant à `1`.
- `value` doit avoir une valeur non vide (pas que des espaces) si la question est requise. Si le type de question est associé à une liste d'option, la valeur doit correspondre à une des valeurs de `idx` de la liste d'option.

Table `option_lists` :

- `id` est auto-incrémenté.
- `name` doit avoir au minimum une longueur de 3 caractères.
- Le couple `owner`, `name` doit être unique, autrement dit deux listes d'options d'un même utilisateur (ou "Système" dans le cas où `owner` est `null`) ne peuvent pas avoir le même nom.

Table `option_values` :

- `idx` doit être supérieur à zéro ; les différents records qui ont la même valeur de `option_list` doivent avoir des valeurs consécutives commençant à `1`.
- `label` doit avoir au minimum une longueur de 3 caractères.

- Le couple `option_list`, `label` doit être unique, autrement dit deux options d'une même liste d'options ne peuvent pas avoir le même label.

Table `distlists` :

- `id` est auto-incrémenté.
- `name` doit avoir au minimum une longueur de 3 caractères.
- `description`, si elle est remplie, doit avoir au minimum une longueur de 3 caractères.
- Le couple `owner`, `name` doit être unique, autrement dit deux listes de distribution d'un même utilisateur ne peuvent pas avoir le même nom.

Droits d'accès

Comme expliqué ci-dessus, les droits d'accès aux formulaires sont gérés grâce aux tables `user_form_accesses` et `distlist_form_accesses`.

Voici les règles qui régissent ces droits d'accès :

- Tous les utilisateurs, y compris les utilisateurs anonymes, peuvent accéder aux formulaires publics en mode d'ouverture pour y répondre. En revanche, l'accès aux formulaires publics en mode d'édition est régi, comme pour les formulaires non publics, par les règles qui suivent.
- Le propriétaire d'un formulaire peut réaliser toutes les opérations valides sur son formulaire.
- Un admin a accès à tous les formulaires et peut y réaliser toutes les opérations valides que peut réaliser le propriétaire du formulaire.
- Un utilisateur a accès à un formulaire dont il n'est pas propriétaire pour autant qu'il ait reçu un droit d'accès au formulaire (table `user_form_accesses`) ou qu'il fasse partie d'une liste de distribution qui a reçu un droit d'accès au formulaire (table `distlist_form_accesses`).
- Le niveau d'accès ainsi autorisé peut prendre la valeur `'editor'` ou `'user'` (s'il y a plusieurs niveaux d'accès autorisés, on prendra l'accès de niveau plus élevé, c'est-à-dire `'editor'`) :
  - Un niveau d'accès `'editor'` donne le droit de réaliser toutes les opérations valides sur le formulaire, à l'exception de la gestion des partages.
  - Un niveau d'accès `'user'` ne donne le droit que de répondre au formulaire et de visualiser son instance la plus récente.

Suppressions en cascade

Dans la base de données, les relations entre les tables ont été configurées pour qu'elles effectuent des **suppressions en cascade**. Cela signifie que lorsqu'on supprime un enregistrement d'une table, on supprimera en cascade tous les enregistrements des autres tables qui référencent cet enregistrement à travers une clé étrangère.

Par exemple, si l'on supprime un formulaire, il faut également supprimer ses questions, ses instances et leurs réponses, ainsi que les partages correspondants.

Dès lors, une telle suppression doit être déclenchée moyennant une demande de confirmation de l'utilisateur qui précise ce qui va être supprimé.

---

1. L'utilisateur virtuel `guest` ne doit pas pouvoir se connecter à l'application, même si en base de données il y a une valeur pour le mot de passe puisque le champ est obligatoire. ↩